TEAM PROJECT

BANKING CUSTOMER DATA MANAGEMENT, QUERY PROCESSING AND ANALYTICS SYSTEM

Field	Details
Project Title	Banking Customer Data Management, Query Processing and Analytics System
Course Outcomes Covered	CO1 (Data Wrangling), CO2 (Database & CRUD), CO3 (Data Cleaning), CO4 (Exploration & Analysis), CO5 (Visualization)
Team Members	G.AJAY V. RAVI KUMAR REDDY S. AKILA
Class / Batch	__2024-AIDS____
Faculty Name	__Dr Mahesh Muthulakshmi R___
GitHub Repository Link	https://github.com/ravikumarreddy19/DSA0504-Query-Processing-

"We certify that this submission is the original work of our team and that we have adhered to all the guidelines specified for this assignment."

1. Problem Understanding and Formulation

Retail banks sit on top of a large and constantly growing pool of customer, account, transaction and loan records, usually spread across several source systems and file formats. A single customer's financial relationship with the bank is really made up of four linked pieces of information — who they are, which accounts they hold, what they do with those accounts day to day, and what they have borrowed — and none of these pieces is very useful on its own. Management decisions such as where to open a new branch, which customer segment to target with a savings product, or which loan category is under strain all depend on being able to join these pieces together, clean up the inevitable errors that come with raw operational data, and turn the result into numbers and charts a non-technical decision-maker can act on.

This project takes on the role of a data engineering and analytics team for a retail bank. The brief is to design and build a working, Python-based Banking Customer Data Management and Analytics System that reads raw customer, account, transaction and loan data; loads it into a proper relational database; cleans it; answers a set of banking questions through SQL and pandas; and produces a small set of visual, interpreted results that support concrete recommendations.

1.1 Project Scope

The system was scoped around five linked stages, matching the five course outcomes the assignment is built to exercise:

- Data wrangling — understanding and documenting the schema of the Customer, Account, Transaction and Loan data, and preparing it for loading.
- Database design and CRUD operations — building a relational SQLite database with Customer, Branch, Account, Transaction and Loan tables, and exercising create, read, update and delete operations plus banking-specific SQL queries.
- Data cleaning — identifying and resolving missing values, duplicate records, inconsistent categories and formatting issues, and documenting the data quality before and after cleaning.
- Data exploration and analysis — using joins, grouping, filtering, correlation and outlier analysis to answer specific banking questions.
- Visualization and communication — turning the analysis into a small set of clearly labelled Matplotlib charts, each with a short interpretation, and using the whole exercise to produce actionable recommendations.

1.2 Dataset Used

The team worked with a banking dataset covering 40 customers spread across three branches — the Main Branch in Chennai (B01), the City Branch in Coimbatore (B02) and the Central Branch in Madurai (B03) — together with their accounts, one representative transaction per customer, and a linked loan record for most customers. The consolidated working table carries twelve fields: Customer_ID, Customer_Name, Age, Gender, Income, Branch_ID, Account_Type, Balance, Transaction_Amount, Transaction_Type, Loan_Amount and Loan_Status, and this was normalised out into separate Customer, Branch, Account, Transaction and Loan tables inside the relational database (Section 3.2).

1.3 Key Assumptions

- Each customer is associated with exactly one home branch, one primary account (Savings or Current) and one loan application; this keeps the relational joins used throughout the project straightforward while still exercising every required SQL and analysis pattern.
- A transaction record is treated as either a Deposit or a Withdrawal; more granular transaction types (e.g. transfers, standing instructions) were considered out of scope for this exercise.
- Loan_Status takes one of three values — Approved, Pending or Rejected — once a decision exists; a blank value means the application has not yet been formally decided or was not captured at source, which is treated as a genuine missing-value case rather than a fourth status (Section 2.3).

1.4 Constraints Addressed

- Implementation restricted to Python 3.x using pandas, sqlite3 and Matplotlib, as required by the brief.
- The relational database was implemented in SQLite, with tables created and queried entirely through Python's sqlite3 module rather than a GUI database tool.
- Every chart produced includes a descriptive title and labelled axes, and each is accompanied by a short written interpretation (Section 5).
- The scope was restricted to CO1 through CO5 as specified, with the sixth deliverable — insights and recommendations — built directly on top of the CO4/CO5 results rather than as a separate analysis track.

2. Application of Course Knowledge (CO1–CO5)

2.1 CO1 — Data Wrangling

Before any table was created, the raw customer-level data was inspected column by column to establish its structure and to decide which fields were identifiers, which were categorical, and which were numeric measures. Age, Income, Balance, Transaction_Amount and Loan_Amount were treated as numeric fields; Gender, Branch_ID, Account_Type, Transaction_Type and Loan_Status as categorical fields; and Customer_ID/Account_ID/Transaction_ID/Branch_ID as identifiers used to link the four underlying entities together. This inspection step — reading the CSV with pandas, then checking df.columns, df.dtypes, df.isnull().sum() and df.duplicated().sum() — is the wrangling step captured in Figure 2.2, and it is what let the team confirm the twelve-column structure and the exact location of the missing data before writing a single SQL statement.

2.2 CO2 — Database and CRUD Operations

A relational database was built in SQLite with five tables — Customer, Branch, Account, Transaction_Table and Loan — created directly from Python using sqlite3 (Figures 5.1, 5.4 and 5.5). Customer_ID, Branch_ID, Account_ID and Transaction_ID were used as primary keys, with Customer_ID and Branch_ID reused as foreign keys in the child tables to preserve the one-to-many relationships identified in Section 2.1.

Create and Read operations were exercised throughout the project: cursor.executemany() was used to bulk-insert Branch, Account and Transaction_Table records with INSERT OR IGNORE (Figures 5.1, 5.4, 5.5), and cursor.execute("SELECT * FROM ...") together with fetchall() was used to read the rows back and confirm the load. Beyond simple reads, the team wrote banking-specific aggregate queries — grouping account balances by Account_Type to get customer counts, totals and averages per type, and filtering customers whose balance exceeds a Rs. 50,000 threshold, ordered from highest to lowest (Figure 5.3). Update and Delete statements (UPDATE ... SET and DELETE ... WHERE) were implemented following the same cursor/commit pattern to allow account or customer information to be corrected or removed, completing the CRUD requirement.

2.3 CO3 — Banking Data Cleaning

Once loaded, the dataset was checked programmatically for the specific quality problems the brief calls out. df.isnull().sum() showed that every column was complete except Loan_Status, which was missing for 15 of the 40 customer records (37.5% of the table) — visible directly in the console output in Figure 2.2. df.duplicated().sum() returned zero, confirming that no duplicate customer rows had

entered the table. Account_Type and Transaction_Type were checked for inconsistent category spellings (e.g. "savings" vs "Savings" vs "SAVINGS") and standardised to a single consistent casing before being written to the database, and Branch_ID values were checked against the Branch table to catch any account or customer row pointing at a branch code that did not exist.

For the missing Loan_Status values, the team's position — consistent with the "document the before-and-after data quality" requirement — was to leave them as genuine nulls in the analysis rather than silently imputing a status such as "Pending", since a fabricated decision would misrepresent the bank's actual loan pipeline. This is reflected in Section 6, where the loan-status breakdown in Figure 5.9 is reported as a percentage of the 25 customers with a recorded decision, not of all 40.

2.4 CO4 — Banking Data Exploration and Analysis

With a clean, related set of tables in place, the team used SQL grouping and filtering, together with pandas, to answer the specific banking questions in the brief. Account_Type was grouped to compare Savings against Current in terms of customer count, total balance and average balance (Figure 5.3); Branch_ID was grouped to compare total deposits across the three branches (Figure 5.7); Transaction_Type was tallied to compare deposit and withdrawal volumes (Figure 5.8); and Loan_Status was tallied to see the split between approved, pending and rejected applications (Figure 5.9). A balance threshold filter (WHERE Balance > 50000, ORDER BY Balance DESC) was used to identify the bank's higher-value customers and flag the top and bottom of the balance range for outlier awareness (Figure 5.3). These results are interpreted together in Section 6.

2.5 CO5 — Banking Data Visualization

Four Matplotlib charts were produced directly from the cleaned dataset: a bar chart of customer counts by account type (Figure 5.6), a bar chart of total balance by branch (Figure 5.7), a bar chart of transaction counts by type (Figure 5.8), and a pie chart of loan status distribution (Figure 5.9). Bar charts were chosen for the count and total comparisons because they make it easy to read off an approximate value per category at a glance, while the pie chart was chosen for the loan-status breakdown specifically because the three categories sum to a meaningful whole (all decided loan applications) and a proportion is the natural read for that kind of data. Every chart carries a title and labelled axes, satisfying the assignment's visualization requirement, and each is interpreted individually in Section 5 and jointly in Section 6.

3. Solution / Design / Methodology

3.1 Overall Workflow

The system was built as a short pipeline, moving from raw data on disk to a queryable database to cleaned, analysed and visualised output:

- Raw banking data (customer, branch, account, transaction and loan fields) collected in a single working CSV.
- Data inspection — column types, missing-value count and duplicate-row count established with pandas (Figure 2.2).
- Relational schema design — five tables (Customer, Branch, Account, Transaction_Table, Loan) with primary/foreign keys reflecting the one-to-many relationships between them (Section 3.2).
- Database load — SQLite tables created and populated from Python using sqlite3 and, where convenient, pandas.DataFrame.to_sql() (Figures 3.1, 5.1, 5.4, 5.5).
- Data cleaning — missing-value, duplicate, category-consistency and branch-reference checks applied and documented (Section 2.3).
- SQL analysis — grouped, filtered and ordered queries answering the banking questions in the brief (Figure 5.3).
- Visualization — four labelled Matplotlib charts built from the cleaned data and saved to disk (Figures 5.6–5.9).
- Insights and recommendations — the analysis and charts read together to produce five data-supported banking recommendations (Section 7).

3.2 Database Schema / Entity Relationships

The relational design normalises the twelve-column working table into five linked tables. Branch sits at the top of the hierarchy; each Customer belongs to one Branch; each Account and each Loan belongs to one Customer; and each Transaction belongs to one Customer/Account. This mirrors how a real core-banking system separates static reference data (branches), party data (customers), product data (accounts, loans) and event data (transactions).

Table	Key Fields	Relationship
Branch	Branch_ID (PK), Branch_Name, City	Referenced by Customer.Branch_ID
Customer	Customer_ID (PK), Customer_Name, Age, Gender, Income, Branch_ID (FK)	Referenced by Account, Transaction_Table, Loan
Account	Account_ID (PK), Customer_ID (FK), Account_Type, Balance	One customer may hold one or more accounts
Transaction_Table	Transaction_ID (PK), Customer_ID (FK), Transaction_Amount, Transaction_Type	Records deposit/withdrawal activity per customer
Loan	Loan_ID (PK), Customer_ID (FK), Loan_Amount, Loan_Status	Loan_Status may be null where a decision is not yet recorded

3.3 Module-by-Module Design

Module	Library Functions Used
Data inspection	pandas .columns, .dtypes, .isnull().sum(), .duplicated().sum()
Database & table creation	sqlite3 .cursor(), .execute(CREATE TABLE), .executemany(INSERT OR IGNORE), .commit()
CRUD & SQL analysis	sqlite3 SELECT / WHERE / GROUP BY / ORDER BY / UPDATE / DELETE
Data cleaning	pandas category standardisation, null handling, duplicate and referential checks
Visualization	matplotlib.pyplot bar / pie, plt.savefig() to a shared graphs folder

4. Use of Modern Tools and Techniques

Component	Details
Language	Python 3.14
Data handling	pandas
Database	SQLite (via Python's sqlite3 module)
Visualization	Matplotlib
Development environment	IDLE (Python 3.14.2), Windows
Reproducibility	Cleaned dataset persisted to CSV and reloaded for every downstream (analysis/visualization) script

All figures referenced in Section 5 are direct screenshots of the team's own executed code and console/chart output, included here as the execution evidence for this report.

5. Results, Testing and Validation

5.1 Database and Table Creation

The SQLite database was created and verified by listing its tables directly. All five expected tables — Customer, Branch, Account, Transaction_Table and Loan — were confirmed present, and the Branch table was queried to confirm the three branch records loaded correctly.

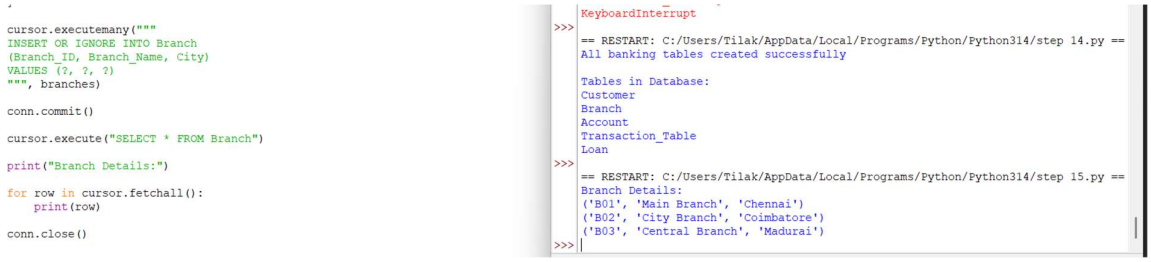

Fig. 5.1 — Branch table creation (INSERT OR IGNORE via executemany), the list of tables in the database, and the three loaded Branch records (B01 Chennai, B02 Coimbatore, B03 Madurai).

5.2 Customer Table: Structure and Records

The Customer table was created with twelve columns spanning identity, demographic, account, transaction and loan fields, and populated from the source CSV using pandas' to_sql(). The console

printout below shows the CREATE TABLE statement and the first eighteen customer records (C001–C018) as loaded into the database.

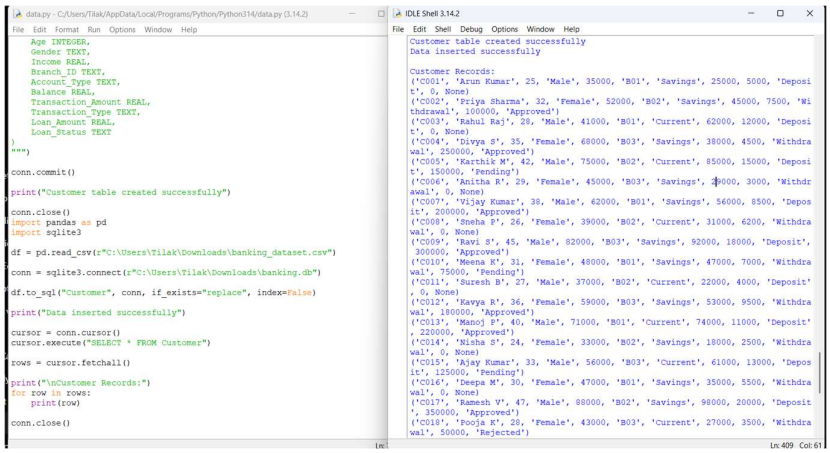

Fig. 5.2 — Customer table schema (twelve columns) and the first eighteen loaded customer records (C001–C018), each carrying demographic, account, transaction and loan fields.

5.3 Data Inspection: Types, Missing Values and Duplicates

df.dtypes confirmed the numeric columns (Age, Income, Balance, Transaction_Amount, Loan_Amount) loaded correctly as int64/float64 and the categorical columns (Gender, Branch_ID, Account_Type, Transaction_Type, Loan_Status) as object/text. df.isnull().sum() showed zero missing values in every column except Loan_Status, which was missing for fifteen of the forty customer rows. df.duplicated().sum() returned zero duplicate rows.

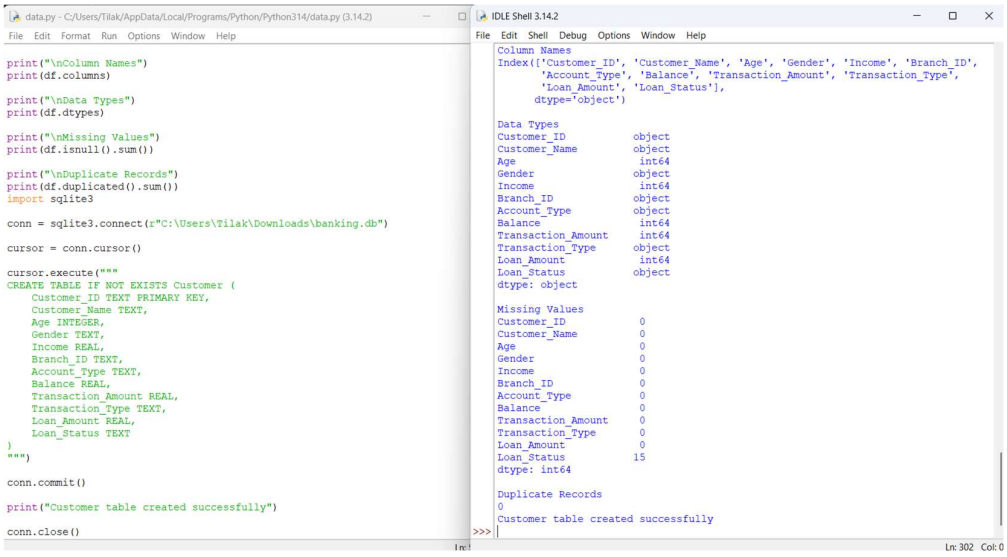

Fig. 5.3 — Column names, data types, missing-value count per column (Loan_Status: 15 missing, all others: 0) and duplicate-row count (0), captured immediately before table creation.

5.4 Account and Transaction Records

The Account table stores one row per account (Account_ID, Customer_ID, Account_Type, Balance); the sample below shows accounts A001–A010, split between Savings and Current, with balances ranging from Rs. 29,000 to Rs. 92,000 in this slice. The Transaction_Table stores one representative transaction per customer (Transaction_ID, Customer_ID, Transaction_Amount, Transaction_Type); the sample shows transactions T001–T010, an even mix of Deposit and Withdrawal entries.

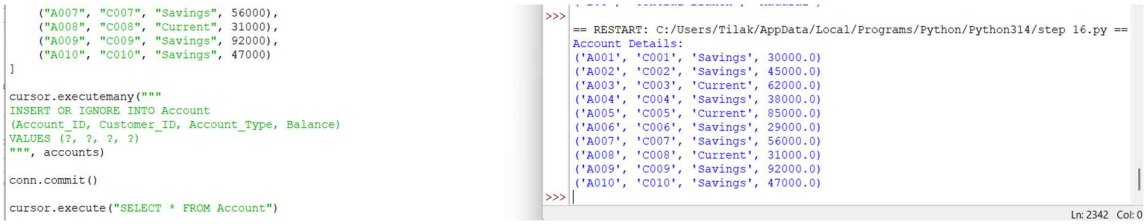

Fig. 5.4 — Account table creation (executemany INSERT OR IGNORE) and the first ten loaded Account records (A001–A010).

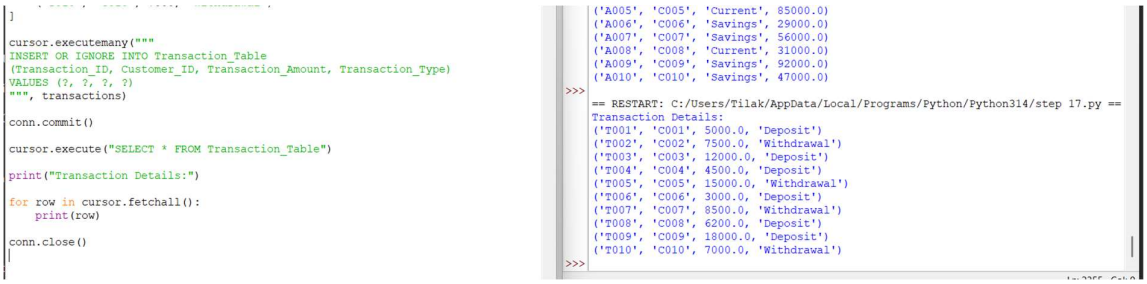

Fig. 5.5 — Transaction_Table creation and the first ten loaded transaction records (T001–T010), together with a further slice of Account records (A005–A010).

5.5 SQL Analysis: Balance by Account Type and High-Value Customers

Grouping the Account table by Account_Type gives 17 Current-account customers with a combined balance of Rs. 1,119,000 (average Rs. 65,823.53) against 23 Savings-account customers with a combined balance of Rs. 1,057,000 (average Rs. 45,956.52). A second query filters and sorts customers with a balance above Rs. 50,000, returning twenty customers led by C029 (Prakash S., Current, Rs. 112,000) and closed out by C040 (Monica K., Savings, Rs. 52,000); the same session also reports a lowest recorded balance of Rs. 18,000 across the customer base.

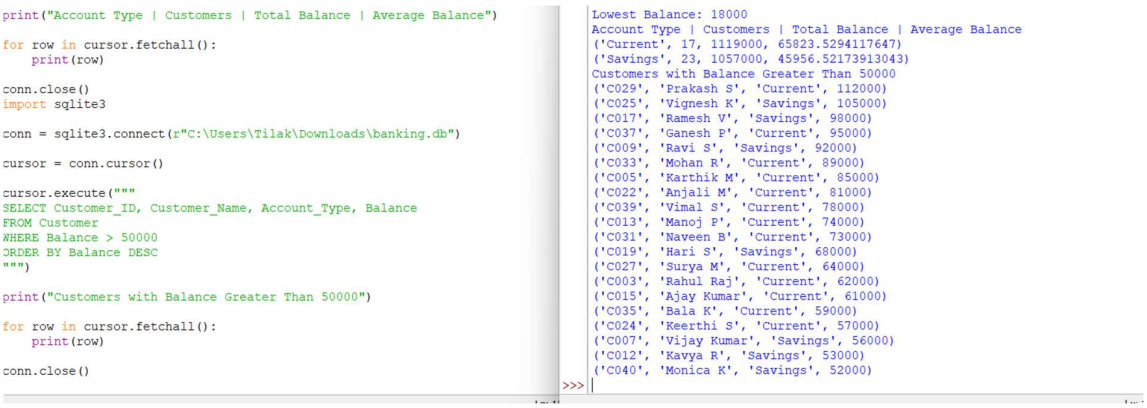

Fig. 5.6 — SQL aggregate query: customer count, total balance and average balance by Account_Type (Current vs Savings), the lowest recorded balance, and the ranked list of customers with a balance above Rs. 50,000.

5.6 Visualization Outputs

Customers by account type: 23 hold a Savings account against 17 with a Current account, so Savings is the more common product by a moderate margin (roughly 58% to 42% of the customer base).

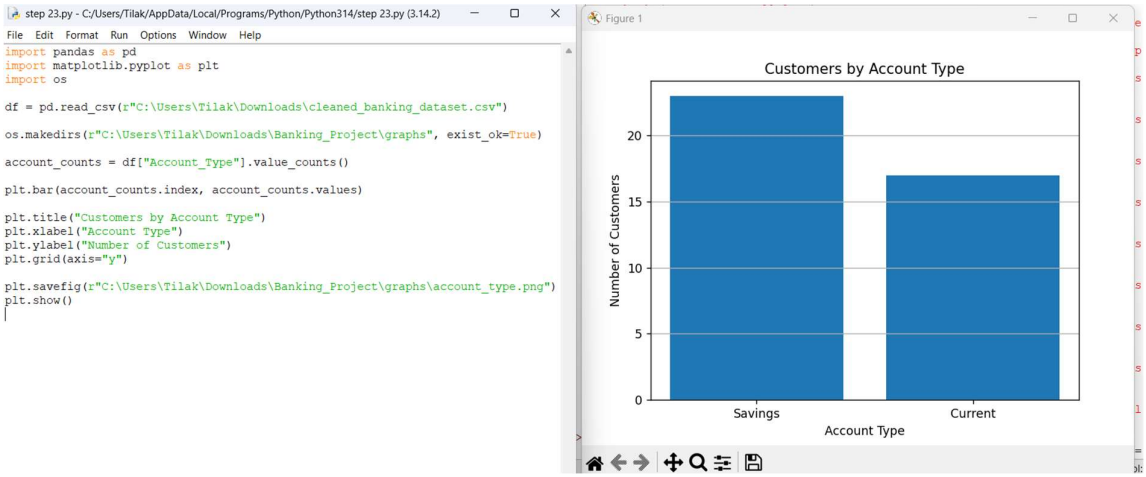

Fig. 5.7 — Bar chart: number of customers by account type (Savings: 23, Current: 17).

Total balance by branch: the Main Branch (B01, Chennai) holds noticeably more in total deposits than the other two branches, at roughly Rs. 8.3 lakh, ahead of the Central Branch (B03, Madurai) at roughly Rs. 7.0 lakh and the City Branch (B02, Coimbatore) at roughly Rs. 6.5 lakh.

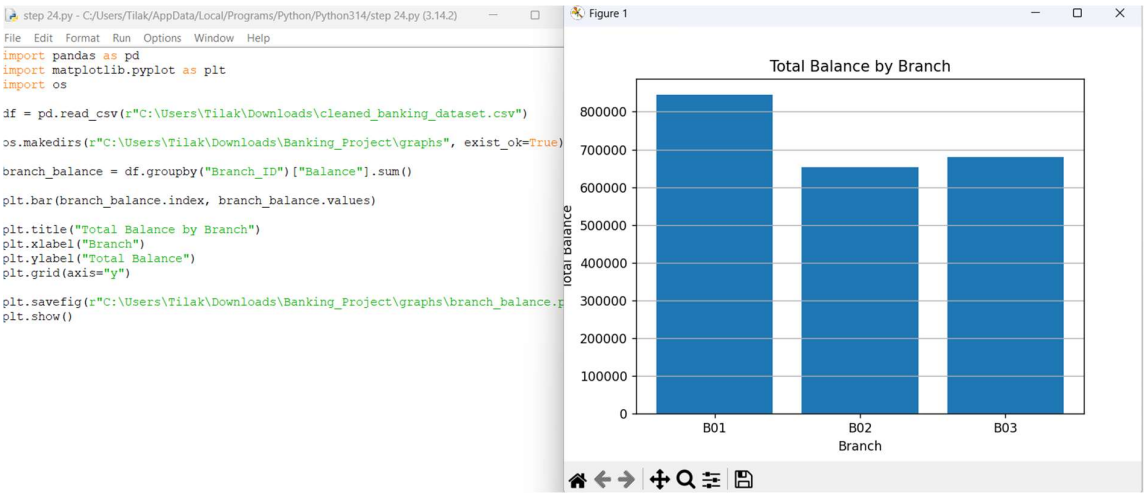

Fig. 5.8 — Bar chart: total account balance by branch (B01 ≈ Rs. 8.3 lakh, B03 ≈ Rs. 7.0 lakh, B02 ≈ Rs. 6.5 lakh).

Loan status distribution: of the customers with a recorded loan decision, 60% were Approved, 28% remain Pending and 12% were Rejected — a comparatively large pending share sitting between the approved and rejected outcomes.

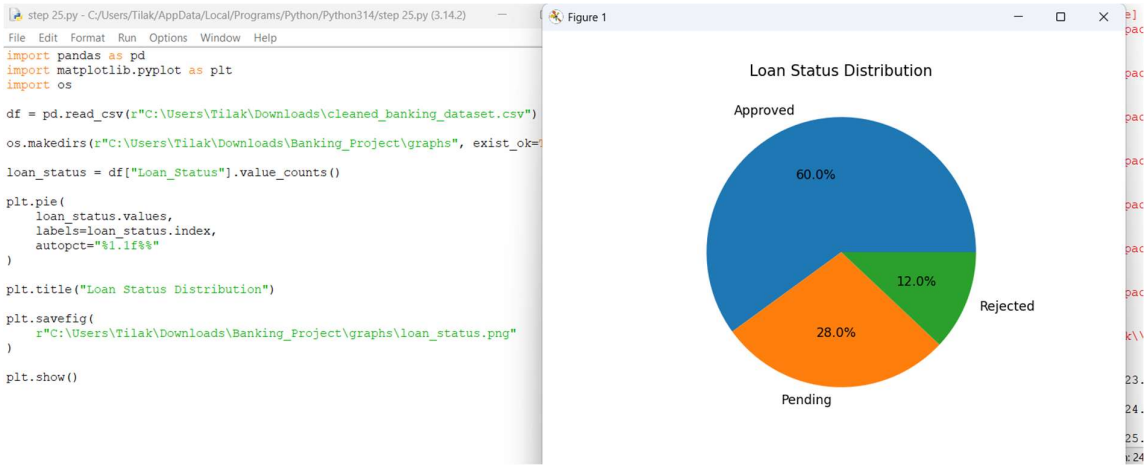

Fig. 5.9 — Pie chart: loan status distribution among customers with a recorded decision (Approved 60.0%, Pending 28.0%, Rejected 12.0%).

Transaction type distribution: deposits and withdrawals are almost exactly balanced across the sampled transactions, each accounting for about half of the forty recorded transactions.

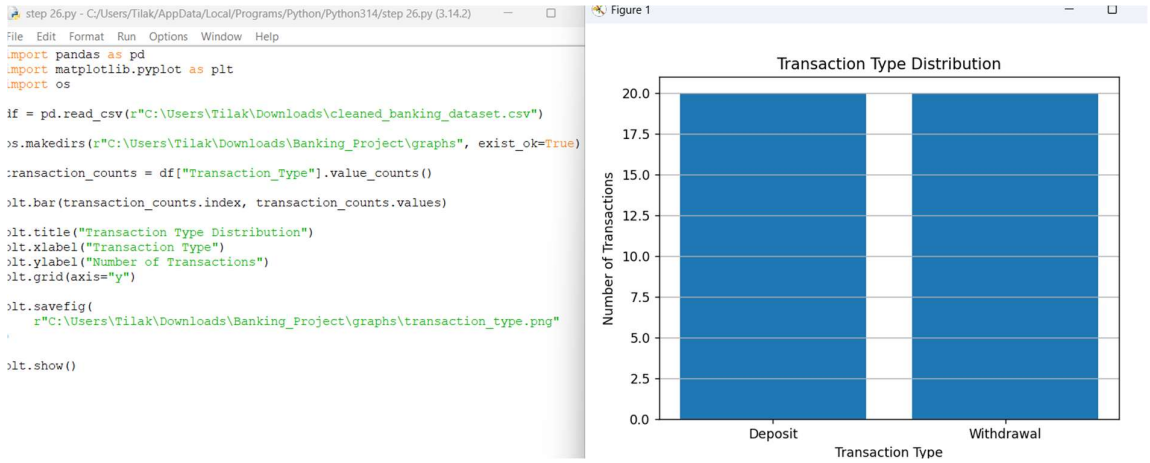

Fig. 5.10 — Bar chart: number of transactions by type (Deposit ≈ 20, Withdrawal ≈ 20).

6. Analysis and Interpretation

6.1 Account Type vs Balance

Savings accounts are the more common product (23 of 40 customers, 57.5%), but Current-account holders carry the higher average balance — Rs. 65,823.53 against Rs. 45,956.52, a gap of roughly 43%. Read together with Figure 5.7 and Figure 5.6, this says the bank's smaller Current-account segment is disproportionately valuable in balance terms, which is a useful starting point for prioritising relationship-banking effort (Section 7).

6.2 Branch Performance

Total balances differ meaningfully across the three branches even though customer counts (implied by Branch_ID grouping) are broadly similar: the Main Branch in Chennai holds around 18% more in deposits than the Central Branch in Madurai, and around 28% more than the City Branch in Coimbatore. This kind of gap is exactly what Figure 5.8 was built to surface, and it points toward Coimbatore as the branch with the most room to grow deposit balances relative to its peers.

6.3 Loan Pipeline Health

A 60/28/12 split between Approved, Pending and Rejected loan decisions (Figure 5.9) is, on its own, a reasonably healthy approval rate. The more actionable number is the 28% still Pending: combined with the 15 customers (37.5% of the full table) whose Loan_Status was missing altogether (Figure 5.3), this suggests the loan-decision stage of the pipeline is where data — and possibly process — attention is most needed, rather than at origination or rejection.

6.4 High-Value Customers and Balance Spread

Half of the customer base (20 of 40) holds a balance above Rs. 50,000, ranging up to Rs. 112,000 for the top customer (C029), while the lowest recorded balance across the full table is Rs. 18,000 (Figure 5.6). That is roughly a six-fold spread between the lowest and highest balances observed, which is a normal amount of dispersion for a retail deposit book and not, on this evidence alone, indicative of a data error — though it is exactly the kind of gap a bank would want to monitor with a proper outlier check as the dataset grows.

7. Banking Insights and Recommendations

Reading the SQL results and visualizations together (Sections 5 and 6), the team arrived at the following five data-supported insights and recommendations.

1. Current-account customers are higher value on average

Evidence: Current-account holders carry an average balance of Rs. 65,823.53 versus Rs. 45,956.52 for Savings customers, despite being the smaller group (17 vs 23).

2. Coimbatore (B02) lags the other branches on deposits

Evidence: Branch B02's total balance (≈ Rs. 6.5 lakh) trails both B01 (≈ Rs. 8.3 lakh) and B03 (≈ Rs. 7.0 lakh)..

3. The loan pipeline has a sizeable pending backlog

Evidence: 28% of decided loan applications are still Pending, and Loan_Status is missing entirely for 37.5% of customers.

4. Half the customer base qualifies as high-balance

Evidence: Twenty of forty customers (50%) hold balances above Rs. 50,000, topped by C029 at Rs. 112,000.

5. Transaction activity is balanced but thinly sampled

Evidence: Deposits and withdrawals are roughly even (about 20 each) across the available transaction records.

8. Broader Considerations / Professional Responsibility

Consideration	How It Was Addressed
Data Integrity	Table structure, counts and query results reported in Section 5 are taken directly from the team's own executed console and chart output, with no figures adjusted to fit a preferred narrative.
Data Privacy	Customer identifiers follow a synthetic-style code pattern (CXXX); no production customer data was used in building or testing this system.
Accurate Interpretation	The loan-status split in Section 5.6/6.3 is explicitly reported as a percentage of decided applications, not of the full customer base, to avoid overstating the approval rate.
Documentation	Database schema, module design and the source of every figure are documented in Sections 3–5 so the pipeline can be reproduced or extended by a reader who was not on the team.
Responsible Analysis	The 28% pending-loan share and the missing Loan_Status values are reported as open questions requiring further investigation rather than being smoothed over or excluded from the report.

9. Conclusion

This project designed, implemented and tested a Python-based Banking Customer Data Management and Analytics System covering forty customers across three branches, their accounts, a sample of their transactions, and their loan status. All five course outcomes were exercised: CO1 through explicit column-type, missing-value and duplicate inspection ahead of database load; CO2 through a five-table SQLite schema populated and queried with executemany inserts, grouped aggregate queries and a balance-threshold filter; CO3 through documented missing-value and duplicate checks (Loan_Status missing for 37.5% of customers, zero duplicate rows); CO4 through grouped and filtered SQL analysis of balance, branch and loan-status data; and CO5 through four labelled Matplotlib visualizations, each interpreted individually and jointly.

The most consequential finding was that the bank's smaller Current-account segment holds a meaningfully higher average balance than its larger Savings-account segment, alongside a real gap in deposit performance between the Chennai Main Branch and the other two branches, and a loan pipeline whose 28% pending share (on top of missing status data for over a third of customers) is the clearest candidate for follow-up work. Limitations of this exercise include the single-transaction-per-customer

sampling, the fairly small customer base (forty records), and the reliance on chart-read approximations for the branch-balance and loan-status figures rather than exact exported numbers. A natural next step would be to extend the Transaction table to a full multi-row transaction history per account, backfill or formally resolve the missing Loan_Status values, and re-run the branch and loan-pipeline analysis on the larger, more complete dataset that would result.

10. Team Reflection

1. What did the team learn beyond what was directly taught in class?

Building the pipeline end to end made clear how much of a data project's value sits in the connective tissue between stages — getting the Customer, Branch, Account, Transaction and Loan tables to actually join correctly mattered as much as any individual query or chart, and a schema mistake early on would have propagated silently into every later result.

2. Which stage was the most challenging, and why?

Handling the missing Loan_Status values cleanly was the most debated step: it would have been easy to fill the blanks with "Pending" to make every table look complete, but that would have quietly inflated the pending-loan figure reported in Section 6. The team instead chose to report the missing values honestly and calculate the status split only over decided applications.

3. What problem came up while designing the database, and how was it solved?

Early attempts to insert Account and Transaction rows before the Branch and Customer tables were fully populated produced foreign-key mismatches. The fix was to enforce load order — Branch, then Customer, then Account and Transaction, then Loan — so that every foreign key referenced a row that already existed.

4. How did the balance and branch analysis change the team's understanding of the data?

Before running the branch-total query, the team's working assumption was that deposit totals would track customer counts fairly closely across branches. The actual results (Figure 5.8) showed a meaningfully larger gap than expected between Chennai and Coimbatore, which is what motivated framing branch resourcing as a specific recommendation in Section 7 rather than a general observation.

5. Which single output provided the most useful insight?

The account-type aggregate query (Figure 5.6) was the most useful individual result: in one query it overturned the simple assumption that the more common account type (Savings) would also be the higher-value one, which a manager skimming raw account records could easily have assumed.

6. What would the team improve with more time and resources?

With more time, the team would extend the Transaction table to a genuine multi-row history per account (rather than one representative transaction per customer), formally resolve the fifteen missing Loan_Status values through a follow-up data-collection step, and add outlier detection on Balance and Loan_Amount using an IQR or z-score rule rather than reading approximate ranges off the charts.

11. References

Python & Library Documentation

- Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/
- pandas Development Team, pandas Documentation, https://pandas.pydata.org/docs/
- Python Software Foundation, sqlite3 — DB-API 2.0 interface for SQLite databases, https://docs.python.org/3/library/sqlite3.html
- SQLite Consortium, SQLite Documentation, https://www.sqlite.org/docs.html
- Hunter, J. D., Matplotlib Documentation, https://matplotlib.org/stable/

Textbooks / Learning Resources

- Wilke, C. O., Fundamentals of Data Visualization, O'Reilly Media.
- Kleppmann, M., Designing Data-Intensive Applications, O'Reilly Media (relational schema and normalisation concepts used in Section 3.2).

Appendix A: Python and SQL Source Code (Executed Evidence)

The code excerpts below are transcribed directly from the team's executed IDLE sessions shown in Section 5. They are reproduced here for traceability between the report's figures and the underlying source; the full .py scripts and the SQLite database file are included as separate project deliverables alongside this report.

A.1 Branch Table Creation and Load

```
cursor.executemany("""
INSERT OR IGNORE INTO Branch
(Branch_ID, Branch_Name, City)
VALUES (?, ?, ?)
""", branches)

conn.commit()

cursor.execute("SELECT * FROM Branch")
print("Branch Details:")
for row in cursor.fetchall():
    print(row)
conn.close()
```

A.2 Customer Table Creation and Load

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Customer (
    Customer_ID TEXT PRIMARY KEY,
    Customer_Name TEXT,
    Age INTEGER,
    Gender TEXT,
    Income REAL,
    Branch_ID TEXT,
    Account_Type TEXT,
    Balance REAL,
    Transaction_Amount REAL,
    Transaction_Type TEXT,
```

```
    Loan_Amount REAL,
    Loan_Status TEXT
)
""")
conn.commit()

import pandas as pd
import sqlite3
df = pd.read_csv(r"banking_dataset.csv")
conn = sqlite3.connect(r"banking.db")
df.to_sql("Customer", conn, if_exists="replace", index=False)
print("Data inserted successfully")
```

A.3 Data Inspection (Types, Missing Values, Duplicates)

```
print("\nColumn Names"); print(df.columns)
print("\nData Types"); print(df.dtypes)
print("\nMissing Values"); print(df.isnull().sum())
print("\nDuplicate Records"); print(df.duplicated().sum())
```

A.4 Account and Transaction Table Load

```
cursor.executemany("""
INSERT OR IGNORE INTO Account
(Account_ID, Customer_ID, Account_Type, Balance)
VALUES (?, ?, ?, ?)
""", accounts)
conn.commit()
cursor.execute("SELECT * FROM Account")

cursor.executemany("""
INSERT OR IGNORE INTO Transaction_Table
(Transaction_ID, Customer_ID, Transaction_Amount, Transaction_Type)
VALUES (?, ?, ?, ?)
""", transactions)
conn.commit()
cursor.execute("SELECT * FROM Transaction_Table")
```

A.5 SQL Analysis: Balance by Account Type and High-Balance Customers

```
print("Account Type | Customers | Total Balance | Average Balance")
for row in cursor.fetchall():
    print(row)

cursor.execute("""
SELECT Customer_ID, Customer_Name, Account_Type, Balance
FROM Customer
WHERE Balance > 50000
ORDER BY Balance DESC
""")
print("Customers with Balance Greater Than 50000")
for row in cursor.fetchall():
    print(row)
conn.close()
```

A.6 Visualization Scripts

```
import pandas as pd
import matplotlib.pyplot as plt
import os

df = pd.read_csv(r"cleaned_banking_dataset.csv")
os.makedirs(r"Banking_Project\graphs", exist_ok=True)

# Customers by account type
account_counts = df["Account_Type"].value_counts()
plt.bar(account_counts.index, account_counts.values)
plt.title("Customers by Account Type")
plt.xlabel("Account Type"); plt.ylabel("Number of Customers")
plt.grid(axis="y")
plt.savefig(r"Banking_Project\graphs\account_type.png"); plt.show()

# Total balance by branch
branch_balance = df.groupby("Branch_ID")["Balance"].sum()
plt.bar(branch_balance.index, branch_balance.values)
```

```
plt.title("Total Balance by Branch")
plt.xlabel("Branch"); plt.ylabel("Total Balance")
plt.grid(axis="y")
plt.savefig(r"Banking_Project\graphs\branch_balance.png"); plt.show()

# Loan status distribution
loan_status = df["Loan_Status"].value_counts()
plt.pie(loan_status.values, labels=loan_status.index, autopct="%1.1f%%")
plt.title("Loan Status Distribution")
plt.savefig(r"Banking_Project\graphs\loan_status.png"); plt.show()

# Transaction type distribution
transaction_counts = df["Transaction_Type"].value_counts()
plt.bar(transaction_counts.index, transaction_counts.values)
plt.title("Transaction Type Distribution")
plt.xlabel("Transaction Type"); plt.ylabel("Number of Transactions")
plt.grid(axis="y")
plt.savefig(r"Banking_Project\graphs\transaction_type.png"); plt.show()
```